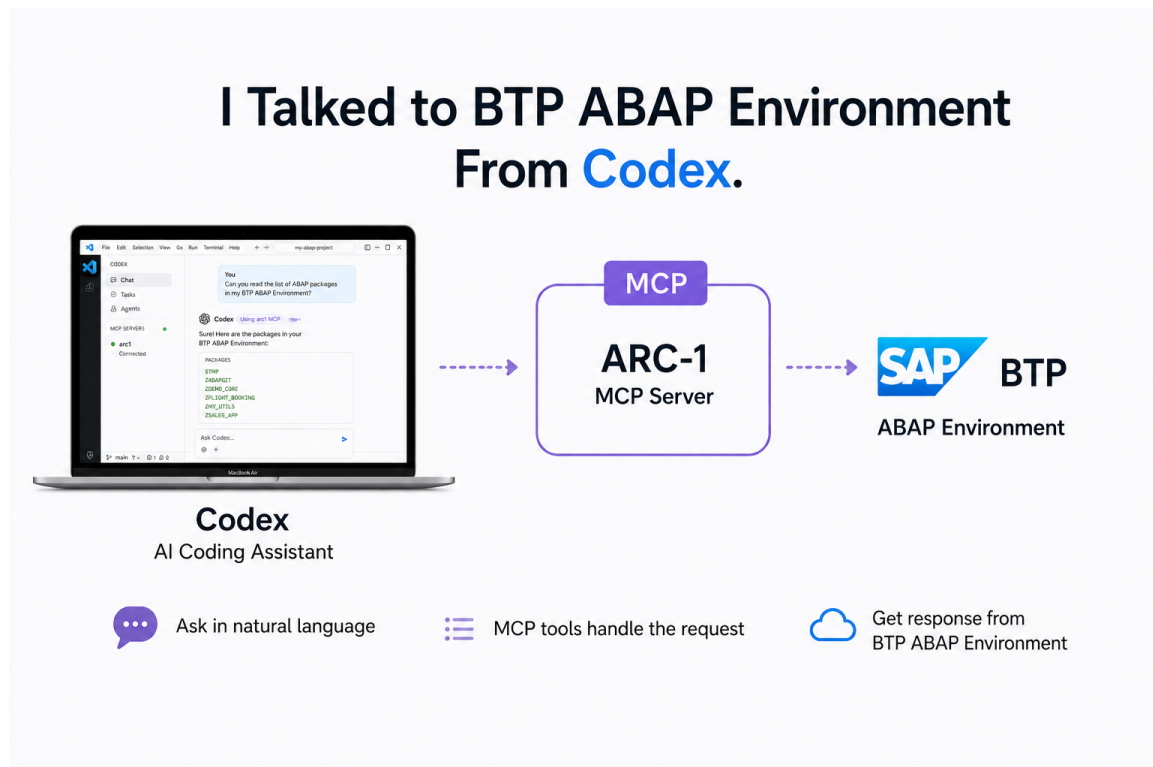


I Talked to BTP ABAP Environment From Codex for Free.

A step-by-step beginner installation and setup guide for ARC-1 on
macOS with SAP BTP authentication



Step 1: Install Node.js and npm

ARC-1 requires Node.js and npm to run. The recommended version is Node.js 22 LTS.

You can install Node.js directly from Terminal using Homebrew.

Step 1.1 — Check if Homebrew Exists

brew --version

If version information appears, continue to the next step. If 'command not found' appears, install Homebrew.

Step 1.2 — Install Homebrew

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

After installation completes, run the commands shown by Homebrew in Terminal.

Step 1.3 — Install Node.js 22

```
brew install node@22
```

Step 1.4 — Add Node.js to PATH

```
echo 'export PATH="/opt/homebrew/opt/node@22/bin:$PATH"' >>
~/.zshrc
source ~/.zshrc
```

Step 1.5 — Verify Installation

```
node -v
npm -v
```

Expected output should display Node.js and npm versions successfully.

Step 2. SAP BTP ABAP Service Key

You need a SAP BTP ABAP service key JSON file before starting ARC-1.

Reference Guide to get trail instance:

<https://marianfoo.github.io/arc-1/btp-abap-environment/>

Step 3 — Install ARC-1

Install ARC-1 globally using npm.

```
npm install -g arc-1
```

This installs the ARC-1 command globally on your machine.

Step 4 — Verify Installation

```
arc1 --version
```

Expected result: ARC-1 version number appears.

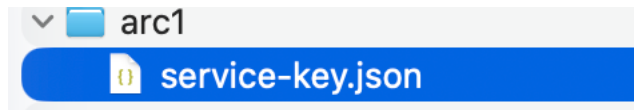
Step 5 — Prepare ARC-1 Workspace

```
mkdir ~/arc1
```

```
cd ~/arc1
```

Copy your SAP BTP service key JSON file into this folder just created and rename it to:

service-key.json



Step 6 — Start ARC-1

```
arc1 --btp-service-key-file service-key.json --transport http-streamable
```

This starts the ARC-1 MCP server using SAP BTP authentication.

Your browser opens automatically for SAP authentication.

Login using your SAP credentials. After successful login, OAuth authentication completes automatically.

Step 7 — Verify ARC-1 Server

Keep the ARC-1 terminal running. Open a second terminal window and run:

```
curl http://localhost:8080/health
```

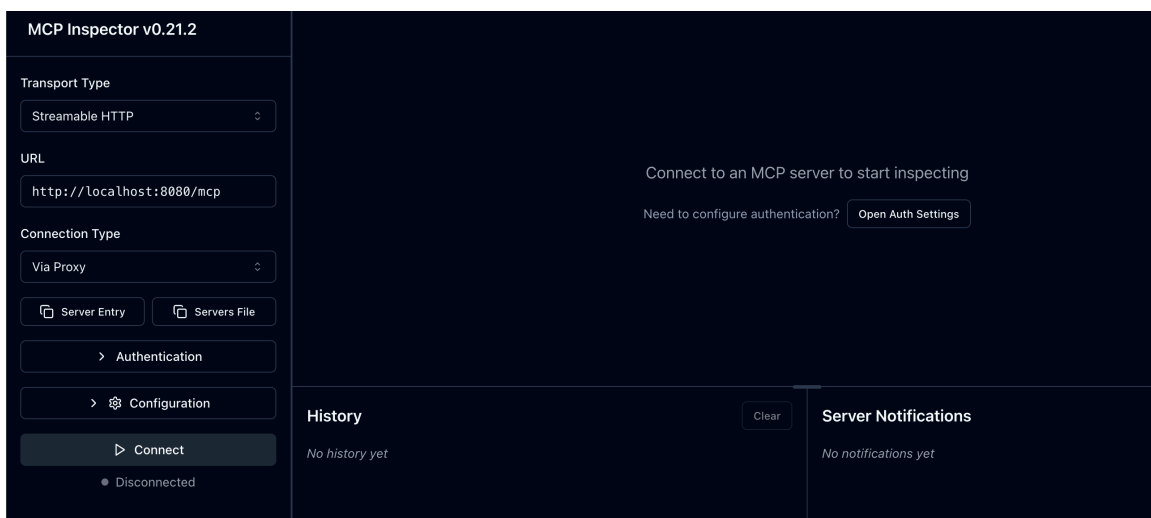
Expected result: JSON response confirming the ARC-1 server is active.

```
{"status":"ok","version":"0.9.5","startedAt":"2026-05-20T18:59:34.678Z","pid":7393}%
```

Step 8 — Install and Start MCP Inspector

```
npx @modelcontextprotocol/inspector
```

This launches the MCP Inspector web UI for testing and debugging MCP servers.



Step 9 — Connect MCP Inspector

Inside MCP Inspector use the following settings:

- Transport: Streamable HTTP
- URL: `http://localhost:8080/mcp`
- Click Connect

Expected result: MCP tools and resources appear successfully.

Step 10 — Install VS code

Open a new terminal window and run:

Since we have already installed home brew, we can run the following command.

`brew install --cask visual-studio-code`

To Launch VS Code run:

`open -a "Visual Studio Code"`

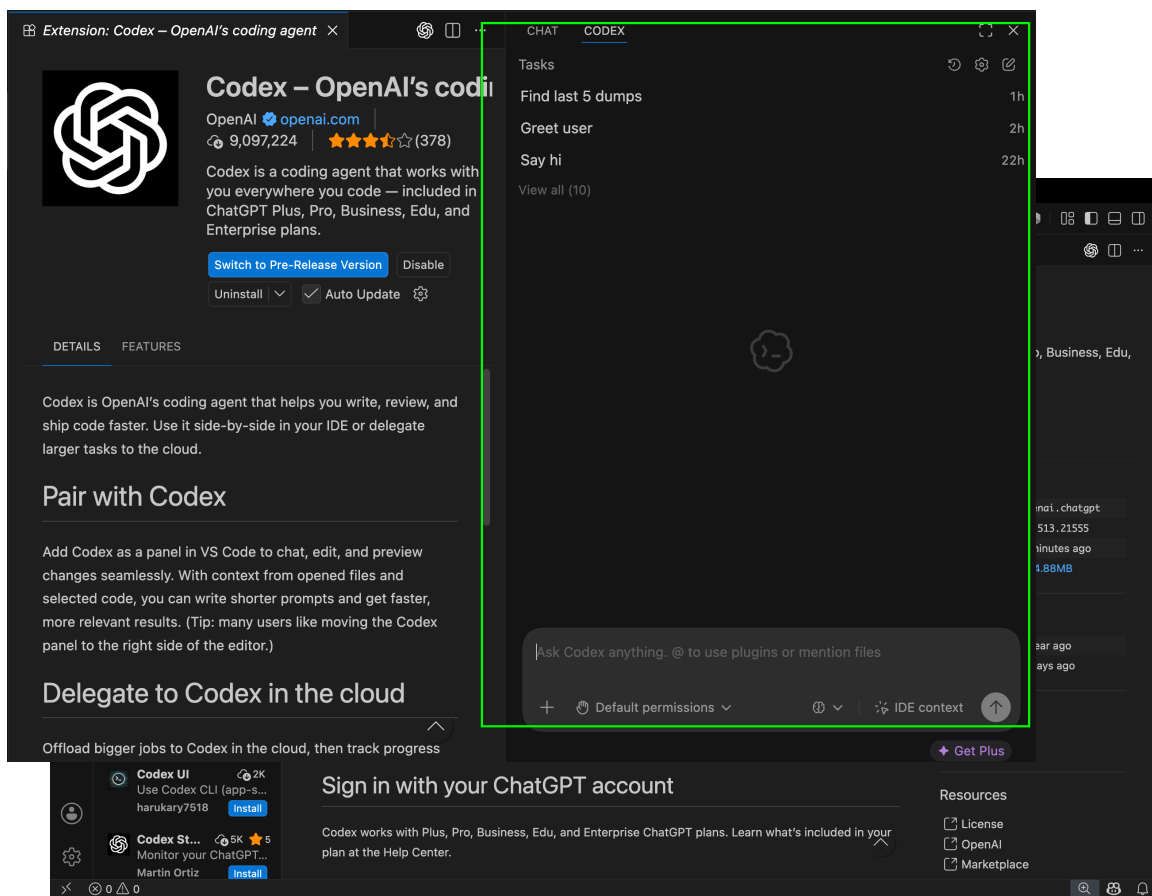
Step 11 — Install Codex Extension

Inside VS Code:

1. Open Extensions (Cmd + Shift + X)
2. Search: OpenAI Codex
3. Install extension
4. Sign in with OpenAI account

After installation:

- a Codex icon appears in the VS Code sidebar
- usually on the left side
- sometimes on the right panel



Click on the Code icon (chatgpt icon) on the right top corner.

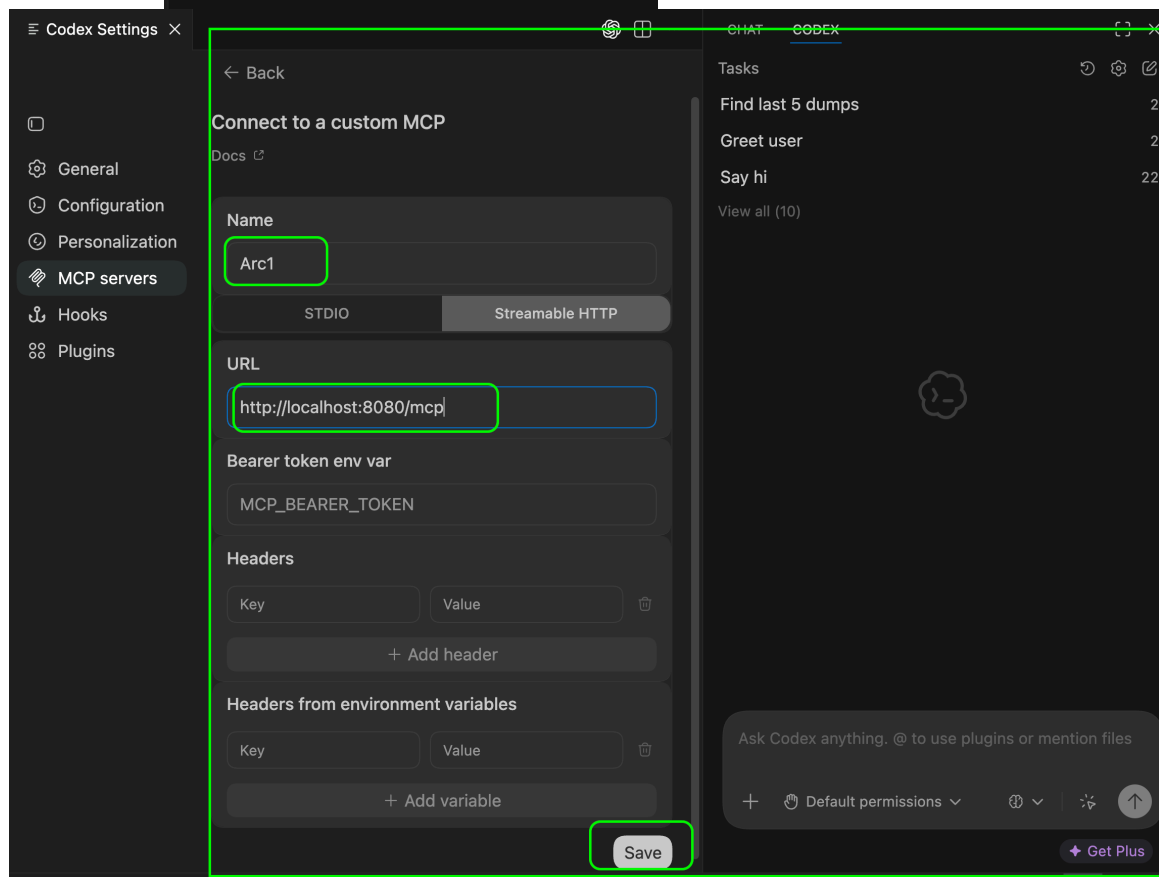
The chat box open for codex. Next step is to configure the MCP server. So that codex can talk to BTP ABAP Environment via MCP server.

Step 12 — Install Codex Extension

Click on settings icon->Codex Settings.

details:

Maintain the following

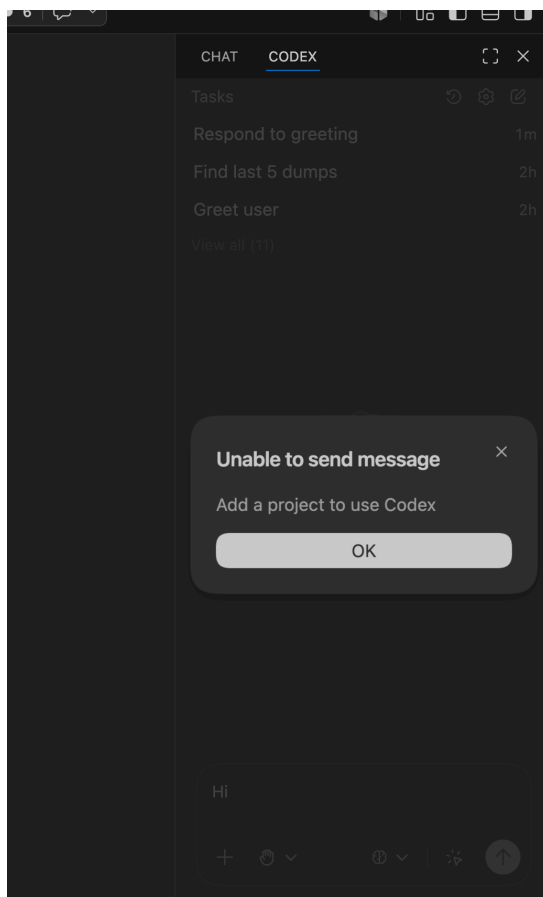


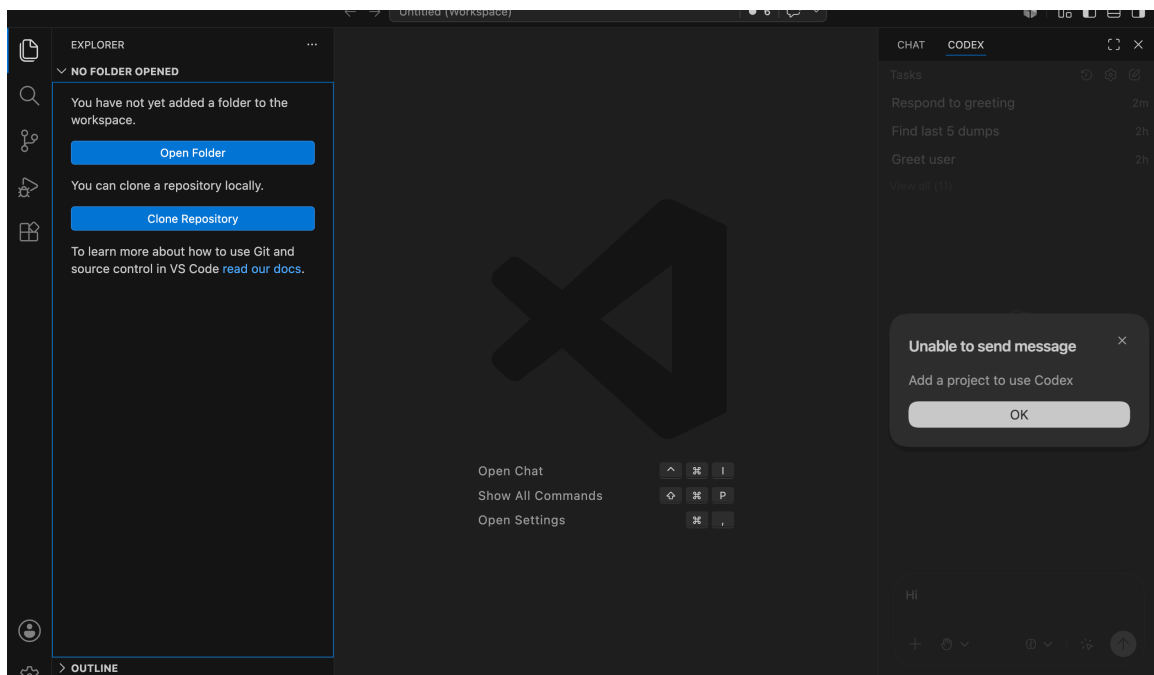
Name: Arc1 , URL: http://localhost:8080/mcp and click on save.

Ensure ARC-1 is already running by checking via step 8 (MCP server)

You are done:

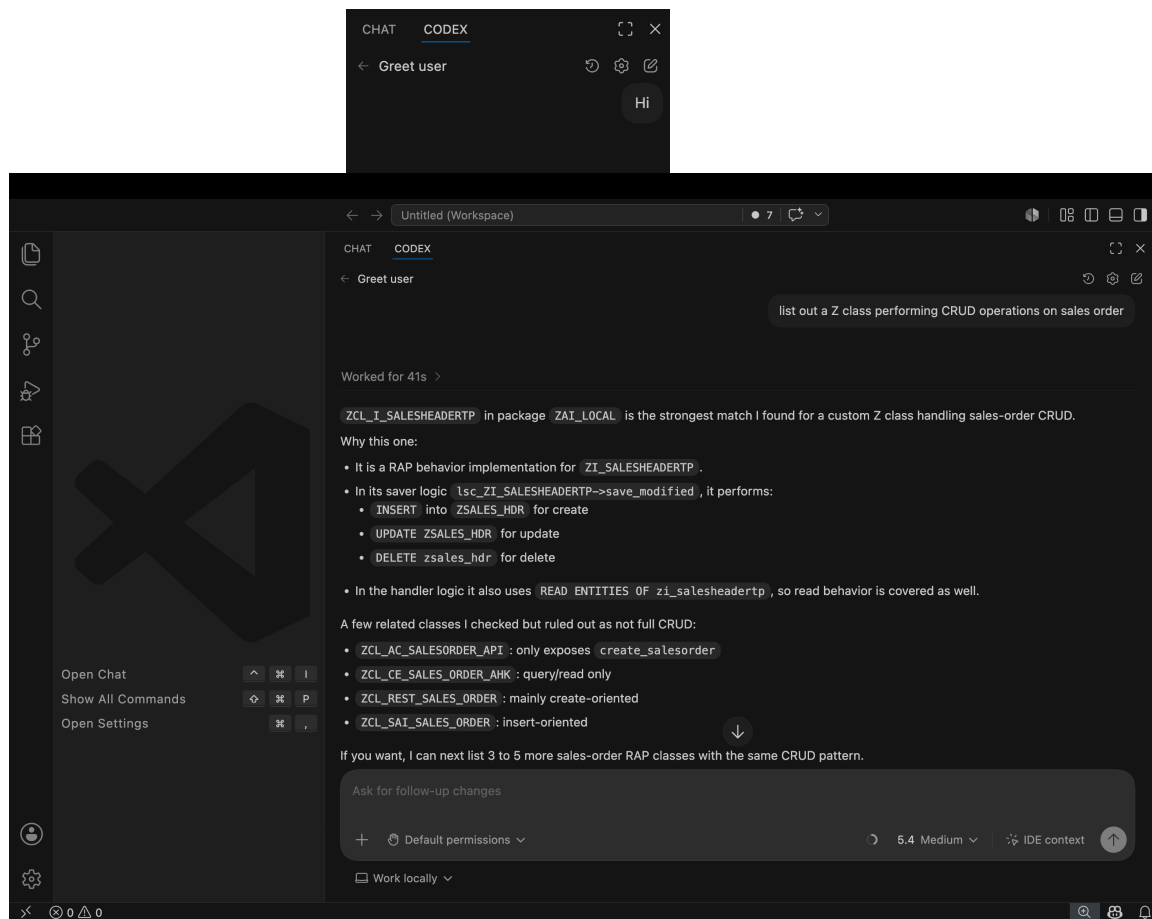
But wait before you talk to it, you need to add a project in VS code.
Or else you get following message.





In the explorer, open folder and choose any folder for your project. You are done.

Say Hi, you get the message. Also you can choose the intelligence. As shown below.



Example conversation:

